Python Interview Questions for AI/ML Engineers Series - 1

Introduction

This document covers the most frequently asked Python interview questions for AI/ML positions at FAANG companies and leading tech startups. Each question includes detailed problem analysis, multiple solution approaches, time/space complexity analysis, and code implementation.

Q1: Reverse a String (with constraints)

Problem Statement:
Write a function to reverse a string. You must handle the case where the string is immutable and cannot be modified in-place.

Example:
Input: s = "hello"
Output: "olleh"

Approach 1: Using Python String Slicing (Pythonic)

```
def reverse_string(s):
return s[::-1]
```

Time Complexity: O(n)

Space Complexity: O(n) - creates a new string

Approach 2: Using a Loop (Interview Preference)

```
def reverse_string(s):
reversed_str = ""
for char in s:
reversed_str = char + reversed_str
return reversed_str
```

Time Complexity: $O(n^2)$ - string concatenation is O(n)

Space Complexity: O(n)

Approach 3: Using List and Join (Efficient)

```
def reverse_string(s):
return "".join(s[i] for i in range(len(s) - 1, -1, -1))
```

Time Complexity: O(n)

Space Complexity: O(n)

Recommended Approach for Interviews: Approach 3

Interview Tips:

- Explain string immutability in Python
- Discuss why list + join is more efficient than string concatenation
- Mention that Python's slicing creates a new object (explain memory implications)

Q2: Find Duplicate in Array

Problem Statement:
Given an array of n+1 integers where each integer is between 1 and n (inclusive), find the duplicate number. You must not modify the array and use only O(1) space.

Example:
Input: nums = [1, 3, 4, 2, 2]
Output: 2

Input: nums = [3, 1, 3, 4, 2]
Output: 3

Approach 1: Hash Set (Simple but violates space constraint)

```
def findDuplicate(nums):
seen = set()
for num in nums:
if num in seen:
return num
seen.add(num)
return -1
```

Time Complexity: O(n)

Space Complexity: O(n)

Approach 2: Cycle Detection (Floyd's Algorithm - Optimal)

```
def findDuplicate(nums):
# Treat array as a linked list where nums[i] points to index nums[i]
slow = fast = nums[0]

# Phase 1: Find intersection point
while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
        break

# Phase 2: Find entrance to cycle
slow = nums[0]
```

```
while slow != fast:
        slow = nums[slow]
        fast = nums[fast]

return slow
```

Time Complexity: O(n)

Space Complexity: O(1)

Approach 3: Negative Marking (When array contains 1 to n)

```
def findDuplicate(nums):
for num in nums:
index = abs(num)
if nums[index] < 0:
return abs(num)
nums[index] = -nums[index]

return -1
```

Time Complexity: O(n)

Space Complexity: O(1)

Note: This modifies the array (not allowed per problem constraints)

Recommended Approach for Interviews: Approach 2 (Floyd's Cycle Detection)

Interview Tips:

- Explain why this is treated as a linked list problem
- Draw a diagram showing the cycle
- Discuss why the cycle must contain the duplicate
- Explain the two-pointer meeting point theorem

Q3: Two Sum Problem

Problem Statement:
Given an array of integers nums and an integer target, return the indices of the two numbers that add up to the target. You may not use the same element twice. Assume each input has exactly one solution.

Example:
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
Explanation: nums[0] + nums[1] = 2 + 7 = 9

Approach 1: Brute Force (Not recommended)

```
def twoSum(nums, target):
for i in range(len(nums)):
for j in range(i + 1, len(nums)):
if nums[i] + nums[j] == target:
return [i, j]
return []
```

Time Complexity: $O(n^2)$

Space Complexity: O(1)

Approach 2: Hash Map (Optimal - Interview Preferred)

```
def twoSum(nums, target):
num_map = {} # value: index

for i, num in enumerate(nums):
        complement = target - num

        if complement in num_map:
        return [num_map[complement], i]

        num_map[num] = i

return []
```

Time Complexity: O(n)

Space Complexity: O(n)

Approach 3: Two Pointers (For sorted array)

```
def twoSum(nums, target):
# Only works if array is sorted
left, right = 0, len(nums) - 1

while left < right:
        current_sum = nums[left] + nums[right]

        if current_sum == target:
        return [left, right]
        elif current_sum < target:
        left += 1
        else:
        right -= 1

return []
```

Time Complexity: O(n log n) for sorting + O(n) for two pointers

Space Complexity: O(1) if sorting is in-place

Recommended Approach for Interviews: Approach 2 (Hash Map)

Interview Tips:

- Explain the trade-off between time and space complexity
- Walk through the logic step-by-step with the example
- Discuss why you iterate only once through the array
- Mention edge cases: empty array, no solution, single element

Q4: Longest Substring Without Repeating Characters

Problem Statement:
Given a string s, find the length of the longest substring without repeating characters.

Example:
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc" with length 3.

Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b" with length 1.

Approach 1: Brute Force
def lengthOfLongestSubstring(s):
max_length = 0

```
for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
        substring = s[i:j]
        if len(substring) == len(set(substring)):
        max_length = max(max_length, len(substring))

return max_length
```

Time Complexity: $O(n^3)$

Space Complexity: O(min(n, m)) where m is charset size

Approach 2: Sliding Window (Optimal)
def lengthOfLongestSubstring(s):
char_index = {} # char: most recent index
max_length = 0
left = 0

```
for right in range(len(s)):
        if s[right] in char_index:
        # Move left pointer to avoid duplicate
        left = max(left, char_index[s[right]] + 1)

        char_index[s[right]] = right
        max_length = max(max_length, right - left + 1)

return max_length
```

Time Complexity: O(n)

Space Complexity: O(min(n, m)) where m is charset size (typically 26 for lowercase)

Walkthrough with Example "abcabcbb":

left=0, right=0: char='a', max_length=1
left=0, right=1: char='b', max_length=2
left=0, right=2: char='c', max_length=3
left=0, right=3: char='a' (repeat), left=1, max_length=3
left=1, right=4: char='b' (repeat), left=2, max_length=3
left=2, right=5: char='c' (repeat), left=3, max_length=3
left=3, right=6: char='b', max_length=4 ← Error, should be 3

Corrected Approach 2:

```
def lengthOfLongestSubstring(s):
char_index = {}
max_length = 0
left = 0

for right in range(len(s)):
        if s[right] in char_index and char_index[s[right]] >= left:
        # Only move left if the repeated char is in current window
        left = char_index[s[right]] + 1

        char_index[s[right]] = right
        max_length = max(max_length, right - left + 1)

return max_length
```

Time Complexity: O(n)

Space Complexity: O(min(n, m))

Interview Tips:

- Explain the sliding window technique
- Draw the window movement on the string
- Discuss why we update the left pointer
- Mention the character-to-index mapping purpose

Q5: Merge K Sorted Lists

Problem Statement:
Merge k sorted linked lists into one sorted linked list.

Example:
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,1,3,4,4,5,6]

Approach 1: Merge Two at a Time
class ListNode:
def **init**(self, val=0, next=None):
self.val = val
self.next = next

def mergeKLists(lists):
if not lists:
return None

```
def merge_two_lists(l1, l2):
        dummy = ListNode(0)
        current = dummy

        while l1 and l2:
        if l1.val <= l2.val:
        current.next = l1
        l1 = l1.next
        else:
        current.next = l2
        l2 = l2.next
        current = current.next

        current.next = l1 if l1 else l2
        return dummy.next

while len(lists) > 1:
        merged_lists = []
        for i in range(0, len(lists), 2):
        l1 = lists[i]
        l2 = lists[i + 1] if i + 1 < len(lists) else None
```

```
        merged_lists.append(merge_two_lists(l1, l2))
            lists = merged_lists

return lists[0] if lists else None
```

Time Complexity: O(N log k) where N is total nodes, k is number of lists

Space Complexity: O(k) for recursion depth

Approach 2: Min Heap (Priority Queue)

```
import heapq

def mergeKLists(lists):
 min_heap = []
 dummy = ListNode(0)
 current = dummy

# Add first node from each list to heap
 for i, lst in enumerate(lists):
            if lst:
            heapq.heappush(min_heap, (lst.val, i, lst))

 while min_heap:
            val, idx, node = heapq.heappop(min_heap)
            current.next = node
            current = current.next

            if node.next:
            heapq.heappush(min_heap, (node.next.val, idx, node.next))

 return dummy.next
```

Time Complexity: O(N log k)

Space Complexity: O(k)

Interview Tips:

- Explain why merging pairs is better than sequential merging
- Discuss heap operations and their complexity
- Mention that tuples in heap need comparison support

Q6: LRU Cache

Problem Statement:
Design a data structure that follows the constraints of a Least Recently Used (LRU) cache. Implement `LRUCache` class with `get()` and `put()` methods.

Constraints:

- Both `get` and `put` must run in O(1) average time complexity

Example:

```
cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
print(cache.get(1)) # Returns 1
cache.put(3, 3) # Evicts key 2
print(cache.get(2)) # Returns -1
```

Approach: Hash Map + Doubly Linked List

```
class Node:
def init(self, key, value):
self.key = key
self.value = value
self.prev = None
self.next = None

class LRUCache:
def init(self, capacity):
self.capacity = capacity
self.cache = {} # key: Node
self.head = Node(0, 0) # Dummy head
self.tail = Node(0, 0) # Dummy tail
self.head.next = self.tail
self.tail.prev = self.head
```

```
def _add_to_head(self, node):
    """Add node right after head (most recent)"""
    node.prev = self.head
    node.next = self.head.next
    self.head.next.prev = node
```

```
        self.head.next = node

def _remove_node(self, node):
        """Remove node from linked list"""
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node

def get(self, key):
        if key not in self.cache:
        return -1

        node = self.cache[key]
        self._remove_node(node)
        self._add_to_head(node)
        return node.value

def put(self, key, value):
        if key in self.cache:
        # Update existing key
        node = self.cache[key]
        node.value = value
        self._remove_node(node)
        self._add_to_head(node)
        else:
        # Add new key
        if len(self.cache) == self.capacity:
        # Remove least recently used (before tail)
        lru_node = self.tail.prev
        self._remove_node(lru_node)
        del self.cache[lru_node.key]

      new_node = Node(key, value)
        self.cache[key] = new_node
        self._add_to_head(new_node)
```

Time Complexity: O(1) for both get and put

Space Complexity: O(capacity)

Interview Tips:

- Explain why doubly linked list is necessary
- Discuss why hash map alone is insufficient
- Draw the structure showing head and tail dummy nodes
- Explain the eviction strategy

Q7: Word Ladder

Problem Statement:
Given two words, `beginWord` and `endWord`, and a word list, find the shortest transformation sequence from `beginWord` to `endWord`. Each step must be a valid word in the list, and you can only change one letter at a time.

Example:
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5
Explanation: "hit" → "hot" → "dot" → "dog" → "cog"

Approach: BFS (Breadth-First Search)

```
from collections import deque

def ladderLength(beginWord, endWord, wordList):
 word_set = set(wordList)

if endWord not in word_set:
        return 0

 queue = deque([(beginWord, 1)])
 visited = {beginWord}

 while queue:
        word, steps = queue.popleft()

        if word == endWord:
        return steps

        # Try all possible one-letter transformations
        for i in range(len(word)):
        for c in 'abcdefghijklmnopqrstuvwxyz':
        if c == word[i]:
        continue

        new_word = word[:i] + c + word[i+1:]

        if new_word in word_set and new_word not in visited:
```

```
        visited.add(new_word)
        queue.append((new_word, steps + 1))

return 0
```

Time Complexity: O(N * L * 26) where N is words, L is word length

Space Complexity: O(N * L)

Optimized Approach: Bidirectional BFS

```
def ladderLength(beginWord, endWord, wordList):
word_set = set(wordList)

if endWord not in word_set:
        return 0

def neighbors(word):
        for i in range(len(word)):
        for c in 'abcdefghijklmnopqrstuvwxyz':
        if c != word[i]:
        yield word[:i] + c + word[i+1:]

# Bidirectional BFS
begin_queue = deque([(beginWord, 1)])
end_queue = deque([(endWord, 1)])
begin_visited = {beginWord}
end_visited = {endWord}

while begin_queue and end_queue:
        # Expand the smaller frontier
        if len(begin_visited) <= len(end_visited):
        if search_level(begin_queue, begin_visited, end_visited, neighbors) != -1:
        return search_level(begin_queue, begin_visited, end_visited, neighbors)
        else:
        if search_level(end_queue, end_visited, begin_visited, neighbors) != -1:
        return search_level(end_queue, end_visited, begin_visited, neighbors)

return 0
```

Time Complexity: O(N * L * 26) - significantly faster in practice

Space Complexity: O(N * L)

Interview Tips:

- Explain why BFS finds shortest path
- Discuss bidirectional BFS optimization
- Explain the word transformation strategy
- Mention why DFS wouldn't be optimal here

Q8: Maximum Product Subarray

Problem Statement:
Given an integer array nums, find a contiguous subarray with the maximum product and return the product.

Example:
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the maximum product 6.

Input: nums = [-2]
Output: -2

Approach: Dynamic Programming
def maxProduct(nums):
if not nums:
return 0

```
max_product = nums[0]
min_product = nums[0]
result = max_product

for i in range(1, len(nums)):
        # Since we have negative numbers, max could become min and min could become max
        max_product, min_product = (
        max(nums[i], max_product * nums[i], min_product * nums[i]),
        min(nums[i], max_product * nums[i], min_product * nums[i])
        )

        result = max(result, max_product)

return result
```

Time Complexity: O(n)

Space Complexity: O(1)

Walkthrough with [2, 3, -2, 4]:

i=0: max=2, min=2, result=2
i=1: max=max(3, 2*3, 2*3)=6, min=min(3, 2*3, 2*3)=3, result=6
i=2: max=max(-2, 6*-2, 3*-2)=-2, min=min(-2, 6*-2, 3*-2)=-12, result=6
i=3: max=max(4, -2*4, -12*4)=4, min=min(4, -2*4, -12*4)=-48, result=6

Interview Tips:

- Explain why we track both max and min
- Discuss why negative numbers complicate the problem
- Walk through an example with negatives
- Mention the simultaneous update using tuples

Q9: Implement Trie (Prefix Tree)

Problem Statement:
Implement a Trie (prefix tree) with `insert()`, `search()`, and `startsWith()` methods.

Example:
```
trie = Trie()
trie.insert("apple")
print(trie.search("apple")) # Returns True
print(trie.search("app")) # Returns False
print(trie.startsWith("app")) # Returns True
```

Approach: Node-Based Trie
```
class TrieNode:
def init(self):
self.children = {}
self.is_end_of_word = False

class Trie:
def init(self):
self.root = TrieNode()
```

```
def insert(self, word):
        node = self.root

        for char in word:
        if char not in node.children:
        node.children[char] = TrieNode()
        node = node.children[char]

        node.is_end_of_word = True

def search(self, word):
        node = self._find_node(word)
        return node is not None and node.is_end_of_word

def startsWith(self, prefix):
        return self._find_node(prefix) is not None
```

```
def _find_node(self, prefix):
        node = self.root

        for char in prefix:
        if char not in node.children:
        return None
        node = node.children[char]

        return node
```

Time Complexity: O(m) where m is word length for each operation

Space Complexity: O(ALPHABET_SIZE * N * M) where N is number of words, M is avg word length

Interview Tips:

- Explain the tree structure and node representation
- Discuss why Trie is better than hash set for prefix queries
- Mention use cases: autocomplete, spell checker, IP routing
- Explain space-time trade-off

Q10: Binary Search Tree Validation

Problem Statement:
Given a binary tree, determine if it is a valid Binary Search Tree (BST).

Example:
Input:
2
/
1 3
Output: True

Input:
5
/
1 4
/
3 6
Output: False (4 is not in right subtree of 5)

Approach 1: In-Order Traversal
class TreeNode:
def **init**(self, val=0, left=None, right=None):
self.val = val
self.left = left
self.right = right

def isValidBST(root):
# In-order traversal of BST gives sorted sequence
in_order = []

```
def traverse(node):
        if not node:
        return
        traverse(node.left)
        in_order.append(node.val)
        traverse(node.right)

traverse(root)
```

```
# Check if in-order traversal is sorted
return in_order == sorted(in_order)
```

Time Complexity: O(n log n) due to sorting

Space Complexity: O(n)

Approach 2: Min-Max Validation (Optimal)

```
def isValidBST(root):
def validate(node, min_val, max_val):
if not node:
return True
```

```
        # Current node must satisfy BST property
        if node.val <= min_val or node.val >= max_val:
        return False

        # Recursively validate left and right subtrees
        return (validate(node.left, min_val, node.val) and
        validate(node.right, node.val, max_val))

return validate(root, float('-inf'), float('inf'))
```

Time Complexity: O(n)

Space Complexity: O(h) where h is height (recursion stack)

Interview Tips:

- Explain the BST property and common mistakes
- Discuss why a naive check (left < parent < right) is insufficient
- Walk through an example with the min-max approach
- Mention the need for valid ranges at each node

Q11: Top K Frequent Elements

Problem Statement:
Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the result in any order.

Example:
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]

Approach 1: Hash Map + Sort
def topKFrequent(nums, k):
from collections import Counter

```
count = Counter(nums)
return [num for num, _ in sorted(count.items(), key=lambda x: x[1], reverse=True)[:k]]
```

Time Complexity: O(n log n)

Space Complexity: O(n)

Approach 2: Min Heap (Optimal for large n, small k)
def topKFrequent(nums, k):
from collections import Counter
import heapq

```
count = Counter(nums)
# Min heap of size k (keeps top k elements)
heap = []

for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
        heapq.heappop(heap)

return [num for freq, num in heap]
```

Time Complexity: O(n log k)

Space Complexity: O(k)

Approach 3: Bucket Sort (Most Efficient)

```
def topKFrequent(nums, k):
from collections import Counter

count = Counter(nums)
# Bucket[i] contains all numbers with frequency i
buckets = [[] for _ in range(len(nums) + 1)]

for num, freq in count.items():
        buckets[freq].append(num)

# Collect k elements from buckets, starting from highest frequency
result = []
for i in range(len(buckets) - 1, 0, -1):
        result.extend(buckets[i])
        if len(result) >= k:
        return result[:k]

return result
```

Time Complexity: O(n)

Space Complexity: O(n)

Interview Tips:

- Explain trade-offs between different approaches
- Discuss when to use heap vs bucket sort
- Explain why bucket sort works for this problem

Q12: Number of Islands

Problem Statement:
Given an m x n 2D binary grid representing a map where '1' represents land and '0' represents water, return the number of islands.

Example:
Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

Approach 1: DFS (Depth-First Search)
def numIslands(grid):
if not grid:
return 0

```
count = 0
rows, cols = len(grid), len(grid[0])

def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0':
        return

        grid[r][c] = '0'  # Mark as visited

        # Explore all 4 directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

for i in range(rows):
        for j in range(cols):
        if grid[i][j] == '1':
        dfs(i, j)
        count += 1
```

```
return count
```

Time Complexity: O(m * n)

Space Complexity: O(m * n) for recursion stack

Approach 2: BFS (Breadth-First Search)

```
from collections import deque

def numIslands(grid):
 if not grid:
 return 0

count = 0
 rows, cols = len(grid), len(grid[0])

 def bfs(start_r, start_c):
        queue = deque([(start_r, start_c)])
        grid[start_r][start_c] = '0'

        while queue:
        r, c = queue.popleft()

        # Explore all 4 directions
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1':
        queue.append((nr, nc))
        grid[nr][nc] = '0'

 for i in range(rows):
    for j in range(cols):
        if grid[i][j] == '1':
        bfs(i, j)
        count += 1

 return count
```

Time Complexity: O(m * n)

Space Complexity: O(min(m, n)) for queue

Interview Tips:

- Explain the island connectivity concept
- Discuss DFS vs BFS trade-offs
- Explain marking as visited to avoid revisiting
- Mention the 4-directional exploration pattern

Q13: Longest Common Subsequence

Problem Statement:
Given two strings `text1` and `text2`, return the length of their longest common subsequence.

Example:
Input: text1 = "ace", text2 = "abc"
Output: 2
Explanation: The LCS is "ac" with length 2.

Approach: Dynamic Programming

```
def longestCommonSubsequence(text1, text2):
m, n = len(text1), len(text2)
# dp[i][j] represents LCS length of text1[0:i] and text2[0:j]
dp = [[0] * (n + 1) for _ in range(m + 1)]

for i in range(1, m + 1):
        for j in range(1, n + 1):
        if text1[i - 1] == text2[j - 1]:
        dp[i][j] = dp[i - 1][j - 1] + 1
        else:
        dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

return dp[m][n]
```

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Space-Optimized Approach:

```
def longestCommonSubsequence(text1, text2):
m, n = len(text1), len(text2)
# Use only two rows instead of 2D array
prev = [0] * (n + 1)
curr = [0] * (n + 1)
```

```
for i in range(1, m + 1):
        for j in range(1, n + 1):
        if text1[i - 1] == text2[j - 1]:
        curr[j] = prev[j - 1] + 1
        else:
        curr[j] = max(prev[j], curr[j - 1])
        prev, curr = curr, prev

 return prev[n]
```

Time Complexity: O(m * n)

Space Complexity: O(n)

Interview Tips:

- Explain the recurrence relation
- Draw the DP table with an example
- Discuss space optimization using rolling arrays
- Mention how to reconstruct the actual subsequence

Q14: Majority Element

Problem Statement:
Given an array nums of size n, return the element that appears more than ⌊n/2⌋ times. Assume the majority element always exists.

Example:
Input: nums = [3,2,3]
Output: 3

Input: nums = [2,2,1,1,1,2,2]
Output: 2

Approach 1: Hash Map
def majorityElement(nums):
count = {}
majority_count = len(nums) // 2

```
for num in nums:
        count[num] = count.get(num, 0) + 1
        if count[num] > majority_count:
        return num
```

Time Complexity: O(n)

Space Complexity: O(n)

Approach 2: Boyer-Moore Voting Algorithm (Optimal)
def majorityElement(nums):
candidate = None
count = 0

```
# Find potential candidate
for num in nums:
        if count == 0:
        candidate = num
        count += 1 if num == candidate else -1
```

```
# Verify candidate (not needed if problem guarantees majority exists)
return candidate
```

Time Complexity: O(n)

Space Complexity: O(1)

Walkthrough with [2,2,1,1,1,2,2]:

num=2: count=0, candidate=2, count=1
num=2: count=2
num=1: count=1
num=1: count=0
num=1: count=0, candidate=1, count=1
num=2: count=0
num=2: count=0, candidate=2, count=1

Interview Tips:

- Explain why Boyer-Moore algorithm works
- Discuss the intuition behind vote cancellation
- Mention why the algorithm guarantees finding the majority
- Explain the verification step if needed

Q15: Coin Change Problem

Problem Statement:
You are given an integer array `coins` and an integer `amount`. Return the fewest number of coins needed to make the amount. Return -1 if impossible.

Example:
Input: coins = [1,2,5], amount = 5
Output: 2
Explanation: 5 = 5 or 5 = 2 + 2 + 1

Approach: Dynamic Programming (Bottom-Up)

```
def coinChange(coins, amount):
# dp[i] represents minimum coins needed for amount i
dp = [float('inf')] * (amount + 1)
dp[0] = 0

for i in range(1, amount + 1):
        for coin in coins:
        if coin <= i:
        dp[i] = min(dp[i], dp[i - coin] + 1)

return dp[amount] if dp[amount] != float('inf') else -1
```

Time Complexity: O(amount * len(coins))

Space Complexity: O(amount)

Approach 2: Dynamic Programming (Top-Down with Memoization)

```
def coinChange(coins, amount):
memo = {}

def dp(remaining):
        if remaining == 0:
        return 0
        if remaining < 0:
        return -1
        if remaining in memo:
```

```
        return memo[remaining]

        min_coins = float('inf')
        for coin in coins:
        result = dp(remaining - coin)
        if result >= 0:
        min_coins = min(min_coins, result + 1)

        memo[remaining] = min_coins if min_coins != float('inf') else -1
        return memo[remaining]

return dp(amount)
```

Time Complexity: O(amount * len(coins))

Space Complexity: O(amount) for memo

Interview Tips:

- Explain the unbounded knapsack variant
- Discuss why greedy approach doesn't always work (counterexample: coins=[3,4], amount=6)
- Draw the DP table with an example
- Mention how to reconstruct the coins used

Q16: Rotate Array

Problem Statement:
Given an integer array `nums`, rotate the array to the right by `k` steps, where k is non-negative.

Example:
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]

Approach 1: Extra Array
def rotate(nums, k):
k = k % len(nums)
rotated = [0] * len(nums)

```
for i in range(len(nums)):
        rotated[(i + k) % len(nums)] = nums[i]

nums[:] = rotated
```

Time Complexity: O(n)

Space Complexity: O(n)

Approach 2: Reverse Algorithm (Optimal)
def rotate(nums, k):
k = k % len(nums)

```
def reverse(start, end):
        while start < end:
        nums[start], nums[end] = nums[end], nums[start]
        start += 1
        end -= 1

reverse(0, len(nums) - 1)
reverse(0, k - 1)
reverse(k, len(nums) - 1)
```

Time Complexity: O(n)

Space Complexity: O(1)

Walkthrough with [1,2,3,4,5,6,7], k=3:
Original: [1,2,3,4,5,6,7]
Reverse all: [7,6,5,4,3,2,1]
Reverse [0:3]: [5,6,7,4,3,2,1]
Reverse [3:7]: [5,6,7,1,2,3,4]

Interview Tips:

- Explain the reverse algorithm intuition
- Discuss why modulo operation is needed
- Walk through the reversals with visual examples
- Mention space complexity advantage

Q17: Remove Duplicates from Sorted Array

Problem Statement:
Given an integer array `nums` sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. Return the number of unique elements.

Example:
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]

Approach: Two Pointer

```
def removeDuplicates(nums):
if len(nums) == 0:
return 0

k = 1  # k is the index where next unique element will be placed

for i in range(1, len(nums)):
        if nums[i] != nums[i - 1]:
        nums[k] = nums[i]
        k += 1

return k
```

Time Complexity: O(n)

Space Complexity: O(1)

Interview Tips:

- Explain why two pointers are used
- Discuss the in-place constraint
- Walk through with the example
- Mention the return value represents length, not index

Q18: Best Time to Buy and Sell Stock

Problem Statement:
Given an array prices where prices[i] is the price on day i, find the maximum profit you can achieve. You can buy and sell at most once.

Example:
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price=1), sell on day 5 (price=6), profit=5

Approach: Single Pass with Min Tracking
def maxProfit(prices):
if not prices or len(prices) < 2:
return 0

```
min_price = prices[0]
max_profit = 0

for price in prices[1:]:
        profit = price - min_price
        max_profit = max(max_profit, profit)
        min_price = min(min_price, price)

return max_profit
```

Time Complexity: O(n)

Space Complexity: O(1)

Interview Tips:

- Explain the greedy approach
- Discuss why we track minimum price
- Walk through with the example
- Mention variations (multiple transactions, with cooldown, etc.)

Q19: Contains Duplicate

Problem Statement:
Given an integer array `nums`, return True if any value appears at least twice in the array.

Example:
Input: nums = [1,2,3,1]
Output: True

Input: nums = [1,2,3,4]
Output: False

Approach 1: Hash Set

```
def containsDuplicate(nums):
return len(nums) != len(set(nums))
```

Time Complexity: O(n)

Space Complexity: O(n)

Approach 2: Hash Set with Early Exit

```
def containsDuplicate(nums):
seen = set()
for num in nums:
if num in seen:
return True
seen.add(num)
return False
```

Time Complexity: O(n) average, O(1) best case

Space Complexity: O(n)

Interview Tips:

- Discuss why sorting approach is suboptimal
- Explain hash set approach

- Mention trade-off between space and time
- Discuss early exit benefit

Q20: Valid Parentheses

Problem Statement:
Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. A string is valid if:

1. Every open bracket has a corresponding close bracket
2. Brackets are closed in the correct order

Example:
Input: s = "({[]})"
Output: True

Input: s = "({[}])"
Output: False

Approach: Stack
def isValid(s):
stack = []
bracket_map = {')': '(', '}': '{', ']': '['}

```
for char in s:
        if char in bracket_map:
        # Closing bracket
        if not stack or stack[-1] != bracket_map[char]:
        return False
        stack.pop()
        else:
        # Opening bracket
        stack.append(char)

return len(stack) == 0
```

Time Complexity: O(n)

Space Complexity: O(n)

Interview Tips:

- Explain why stack is the natural choice
- Discuss the bracket matching with hash map
- Walk through with examples
- Mention edge cases: empty string, only opening, only closing

Interview Preparation Tips

General Strategy

1. **Clarify the problem** - Ask about constraints, edge cases, input format
2. **State assumptions** - Mention what you're assuming about the problem
3. **Discuss approach** - Explain your solution before coding
4. **Analyze complexity** - Always provide time and space complexity
5. **Code carefully** - Write clean, readable code
6. **Test with examples** - Walk through your code with provided examples

Common Pitfalls to Avoid

- Not handling edge cases (empty inputs, single elements)
- Off-by-one errors in loops and indexing
- Not validating input
- Forgetting to handle negative numbers or zero
- Not considering integer overflow

OTT
Opportunity Through Technology